

实验报告

课程名称: Design pattern and software architecture

学 院: 计算机科学与工程学院

专 业: Computer Science and 班 级: SE-7

Technology

姓 名: Muhammad Samudera 学 号:

Bagja

年 月 日

山东科技大学教务处制

实 验 报 告

_____页

组 别	1	姓 名	Samudera	同组实验者	None
实验项目名称	Facade Pattern				
教师评语	Good understanding of the pattern idea and implementation. The report is clear and written in proper English.				
实验成绩:			指导教师（签名）：		
			2026 3 23		

1. Objective

The objective of this experiment is to implement the Facade design pattern using a photobooth session example involving PhotoboothSessionFacade (Facade) and various subsystems (WatermarkEngine, PaymentProcessor, CloudStorage, PrinterService). The goal of this pattern is to provide a unified, simplified interface to a complex system of classes, a library, or a framework.

In this example, the program has the following structure:

- **Facade** (PhotoboothSessionFacade): provides a single, simplified method (processCompleteSession) that orchestrates the execution of multiple subsystem tasks.
- **Subsystems** (WatermarkEngine, PaymentProcessor, CloudStorage, PrinterService): independent classes that perform specific tasks, such as applying frames, processing payments, uploading files to the cloud, and printing.
- **Client** (Application): interacts exclusively with the Facade interface. It does not need to know the details of the subsystems or how they are coordinated — the Facade pattern hides this complexity.

2. Principle

The Facade pattern provides a higher-level interface that makes a subsystem easier to use by wrapping a complex set of classes into a single, simplified interface.

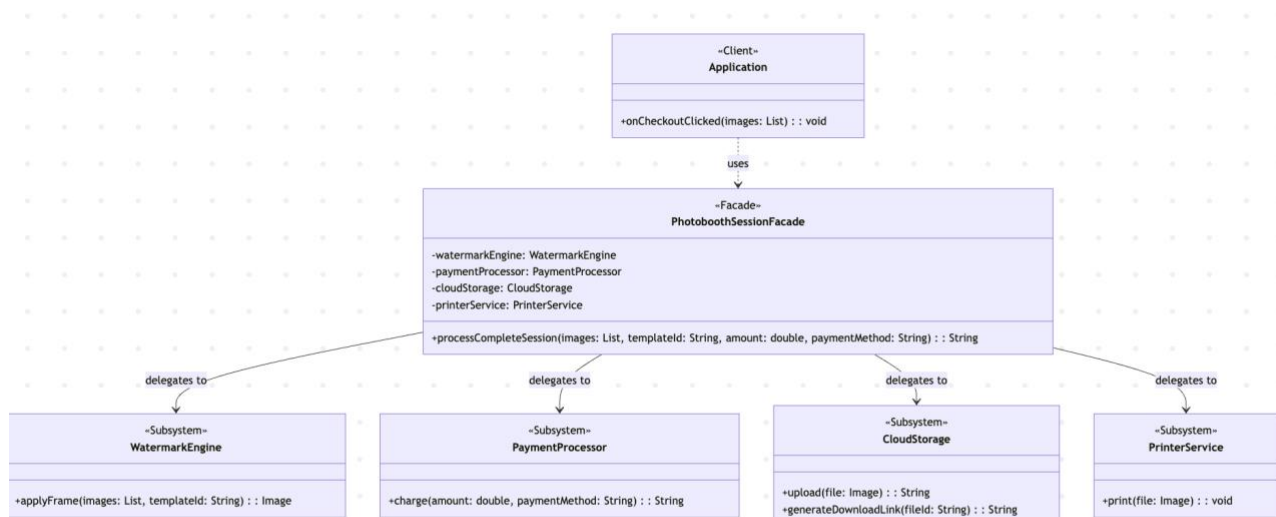
Instead of the client having to manually initialize and coordinate multiple subsystems (e.g., applying a watermark, processing payment, uploading to the cloud, and then printing), the Facade pattern defines a single class (PhotoboothSessionFacade) that delegates these tasks to the appropriate subsystem objects. When the client calls processCompleteSession(), the call is handled internally by

the facade, executing the necessary subsystem methods in the correct order.

Key benefits of this approach include:

- **Simplicity:** The client interacts with a simple interface instead of a complex web of subsystem classes, reducing the learning curve and making the code easier to read.
- **Loose coupling:** The client is decoupled from the subsystems. If the implementation details of a subsystem change, or if a subsystem is replaced, only the facade needs to be updated; the client code remains unaffected.
- **Centralized control:** The facade provides a single point of entry for a specific set of functionalities, making it easier to manage and orchestrate complex workflows.
- **Principle of Least Knowledge (Law of Demeter):** The client only talks to its immediate "friend" (the facade) and does not need to know about the internal structure of the subsystems.

3. UML Diagram



4. Key Code (Java)

```
// Subsystems

class WatermarkEngine {

    public String applyFrame(List<String> images, String templateId) {
```

```

        // logic to apply frame

        return "framed_image_" + templateId + ".jpg";

    }

}

class PaymentProcessor {

    public String charge(double amount, String paymentMethod) {

        // logic to process payment

        return "TXN-" + System.currentTimeMillis();

    }

}

class CloudStorage {

    public String upload(String file) {

        // logic to upload file

        return "FILE-" + file.hashCode();

    }

    public String generateDownloadLink(String fileId) {

        // logic to generate link

        return "https://storage.photobooth.com/download/" + fileId;

    }

}

```

```

class PrinterService {

    public void print(String file) {

        // logic to print

    }

}

// Facade

class PhotoboothSessionFacade {

    private WatermarkEngine watermarkEngine;

    private PaymentProcessor paymentProcessor;

    private CloudStorage cloudStorage;

    private PrinterService printerService;

    public PhotoboothSessionFacade() {

        this.watermarkEngine = new WatermarkEngine();

        this.paymentProcessor = new PaymentProcessor();

        this.cloudStorage = new CloudStorage();

        this.printerService = new PrinterService();

    }

    public String processCompleteSession(List<String> images, String templateId, double amount, String

```

```

paymentMethod) {

    // Step 1: Apply watermark/frame

    String framedImage = watermarkEngine.applyFrame(images, templateId);

    // Step 2: Process payment

    String transactionId = paymentProcessor.charge(amount, paymentMethod);

    // Step 3: Upload to cloud

    String fileId = cloudStorage.upload(framedImage);

    String downloadLink = cloudStorage.generateDownloadLink(fileId);

    // Step 4: Print the image

    printerService.print(framedImage);

    return downloadLink;
}

}

// Client

class Application {

    private PhotoboothSessionFacade facade;

```

```

public Application() {

    this.facade = new PhotoboothSessionFacade();

}

public void onCheckoutClicked(List<String> images) {

    // The client only needs to call one method in the Facade

    // without knowing the details of the subsystems inside

    String downloadUrl = facade.processCompleteSession(

        images, "TEMPLATE_VINTAGE", 25000.0, "QRIS");

}

}

```

5. Analysis

- **Advantages:** The Facade pattern significantly simplifies the client code by providing a unified interface to a complex set of subsystems. In this program, the Application class (client) interacts only with the PhotoboothSessionFacade and calls a single method processCompleteSession() regardless of how many individual steps are required to complete a photobooth session. This eliminates the need for the client to understand or manage the internal workflow of framing, paying, uploading, and printing. It also promotes loose coupling; if a new subsystem is added (e.g., an EmailService to send the download link) or an existing one is changed, the client code remains untouched. The facade encapsulates the complexity and provides a clean, easy-to-use API.
- **Disadvantages:** The Facade pattern can lead to a "god object" if a single facade becomes too large and takes on too many responsibilities, knowing too much about all the classes of an app. While it simplifies the client interface, it does not prevent clients from bypassing the facade and interacting directly with the subsystems if they choose to do so. Furthermore, any changes to the overall workflow might require modifications to the facade class itself, which can become a maintenance bottleneck if not carefully designed.

6. Conclusion

The Facade pattern is well-suited for situations where clients need a simple interface to interact with a complex subsystem, such as in this photobooth session workflow. In this program, PhotoboothSessionFacade acts as the simplified interface, while WatermarkEngine, PaymentProcessor, CloudStorage, and PrinterService represent the internal subsystems that handle

the actual work. By encapsulating these subsystems behind a facade, the system allows complex sessions to be executed through a single method call.

This approach is demonstrated directly in the program structure: the client (Application) only needs to provide the raw images and basic details (template, amount, payment method) to the facade. The facade then orchestrates the entire process, and finally returns the download link to the client. The client class remains completely decoupled from the internal structure and order of operations of the photobooth system.

Overall, the Facade pattern is a practical and scalable solution for managing complex workflows and reducing dependencies between clients and subsystems. It hides the complexity of the subsystem, promotes loose coupling, and keeps the client code clean and maintainable. When interacting with complex libraries, APIs, or sets of classes, the Facade pattern serves as an essential structural tool in software architecture.

